

CONTENTS

- Overview
- 1 The problem both approaches solve
- 2 RAG: the engineering approach
- 3 Long context: the model-native approach
- 4 What changed: three orders of magnitude
- 5 The case for long context: no stack, and no retrieval lottery
- 6 The whole-book problem: reasoning about what is absent
- 7 The case for RAG: you pay to re-read, every single time
- 8 The case for RAG: attention dilutes as context grows
- 9 The case for RAG: the infinite dataset
- 10 Choosing
- Glossary
- Check yourself
- Where to go next

RAG or Long Context: How to Get Data Into a Model

Decide between retrieval and stuffing the context window using cost, recall and data size rather than fashion.

For engineers designing the data path into an LLM application · 26 July 2026

Every section, key point, term and question from the full lesson is here. The extended explanations and the additional examples are not.

[Watch the source video](#)

[Read the full lesson](#)

[Read the highlights](#)

BEFORE YOU START

- What a context window is and that it is finite
- A basic idea of embeddings and vector search
- Comfort reading Python

Overview

Language models are frozen in time. They know the world up to their training cutoff and nothing about what happened five minutes ago, nor anything about your internal wikis or proprietary codebase. Every application that needs current or private data therefore has to solve the same problem: get the right data into the model at the right time. This is context injection.

There are two answers. RAG is the engineering answer — chunk your documents, embed them, store the vectors, search at query time, inject the best matches. Long context is the brute-force answer — put the documents into the prompt and let the model's attention mechanism find what matters. For years the second was impossible because context windows were tiny. They are not tiny any more, which is why the question is live.

The honest answer is that the two fail in different places, and which failure you can afford decides the architecture. RAG can silently fail to retrieve the answer that was sitting right there. Long context makes you pay to re-read everything on every query, dilutes attention as it grows, and hits a hard ceiling long before enterprise data does. Neither is dead; they cover different regimes.

KEY TAKEAWAYS

- ✓ Context injection is the shared problem: a frozen model needs current or private data supplied at query time.
- ✓ RAG chunks and embeds documents ahead of time, then retrieves the closest matches per query and injects them.
- ✓ Long context skips the retrieval stack entirely and lets the attention mechanism do the finding.
- ✓ Long context collapses the infrastructure — no chunking strategy, no embedding model, no vector store, no reranker, no sync.
- ✓ RAG's characteristic failure is silent: the answer existed in the data but retrieval never surfaced it, and nothing reports an error.
- ✓ RAG cannot answer questions about what is absent, because it retrieves what exists and never sees the gap between two documents.
- ✓ Long context re-pays the processing cost on every query; RAG pays it once at index time, and prompt caching only helps for static prefixes.
- ✓ Attention dilutes as context grows, so more context is not monotonically better — and enterprise data will never fit regardless.

01 The problem both approaches solve

0:00

Models are frozen at their training cutoff and have never seen your private data. Everything else follows from that.

KEY POINTS

- Models know nothing past their training cutoff and nothing private to you.
- Context injection is the shared problem: right data, right time, into the prompt.
- Both approaches deliver the data as text in the context window.
- They differ only in whether a selection step happens before the call.
- The trade is the risk of selecting wrongly against the cost of sending everything.

The same call, two ways of filling it

The model sees a prompt either way. The question is what went into the placeholder — four retrieved chunks, or the entire corpus.

```
# RAG: a selection step runs first
context = retrieve(question, k=5)          # ~2,000 tokens

# long context: no selection
context = open("manual.txt").read()       # ~250,000 tokens

# identical from here on
model.generate(f"Context:\n{context}\n\nQuestion: {question}")
```

02 RAG: the engineering approach

0:43

Chunk the documents, embed them, store the vectors, search semantically at query time, inject the winners.

KEY POINTS

- Indexing happens ahead of time: chunk, embed, store.
- Query time is: embed the question, search, inject the top matches.
- The context window ends up holding the prompt plus the retrieved chunks.
- The entire system is conditioned on retrieval having found the right chunks.
- The model cannot know that a better chunk existed and was not retrieved.

The indexing pipeline, once per document

Four stages, all offline. The chunking strategy is a real decision — fixed size, sliding window, or recursive splitting on structure — and it is made before you know what anyone will ask.

```
def index(document, store):
    chunks = chunk(document.text, size=800, overlap=100)
    vectors = embed_batch(chunks)                # one forward pass per chunk

    store.upsert([
        {"doc_id": document.id, "text": c, "vector": v}
        for c, v in zip(chunks, vectors)
    ])

# runs once per document, not per query — this is the whole point
```

03 Long context: the model-native approach

2:10

Skip the database and the embedding model. Put the documents in the prompt and let attention do the heavy lifting.

KEY POINTS

- No embedding model, no vector store, no retrieval step.
- The attention mechanism performs relevance weighting internally, at every layer.
- Retrieval is an approximate external version of what attention already does.
- RAG originated as a workaround for small context windows, not as the better design.
- The model sees everything, so there is nothing for a selection step to miss.

The no-stack stack

The same capability, minus five components. Every box removed is a box that cannot break, drift out of sync, or need capacity planning.

RAG	long context
chunking strategy	—
embedding model	—
vector database	—
reranker	—
index sync pipeline	—
retrieval logic	—
	read the files
model call	model call

04 What changed: three orders of magnitude

2:50

Early context windows held about 4K tokens. Today's frontier models handle a million and more, which is what makes the question worth asking.

KEY POINTS

- Early windows around 4K tokens made RAG mandatory rather than chosen.
- Frontier models now reach a million tokens and beyond.
- A million tokens is roughly 700,000 words of English prose.
- Tokens are not words: code, JSON and non-Latin text are far denser.
- Measure your corpus with a real tokenizer before assuming it fits.

Measure, do not estimate

The rule of thumb is fine for prose and misleading for everything else. Run this over your actual corpus — teams are routinely surprised by a factor of two on codebases and structured data.

```
import tiktoken
enc = tiktoken.get_encoding("cl100k_base")

total = 0
for path in Path("corpus").rglob("*"):
    if path.is_file():
        n = len(enc.encode(path.read_text(errors="ignore")))
        total += n
        print(f"{n:>9,}  {path}")

print(f"{total:>9,}  TOTAL")
print(f"fits in a 1M window: {total < 1_000_000}")
```

05 The case for long context: no stack, and no retrieval lottery

4:20

Two arguments. You delete most of the architecture, and you delete the failure mode that never announces itself.

KEY POINTS

- A production RAG system is chunking, embedding, storage, reranking and sync — five things that can break.
- Long context reduces the data path to reading files and calling the model.
- Semantic search is probabilistic and can miss documents that are present.
- Silent failure: the answer existed, retrieval missed it, nothing reported an error.
- With no retrieval step, long context cannot fail this way at all.

What a silent failure looks like from the outside

This is the entire problem. Both paths return a fluent answer and a 200 status. The only way to tell them apart is to already know the answer — which is why silent failure needs a deliberate measurement rather than monitoring.

```
question: "What is the escalation path for a P1 outage after midnight?"
```

```
retrieved:
```

- [1] "Escalation policy overview ..." ← topical, wrong section
- [2] "P2 and P3 handling during business..." ← topical, wrong severity
- [3] "On-call rotation calendar ..." ← topical, no procedure

```
answer: fluent, confident, wrong
```

```
logs: no errors
```

```
status: 200 OK
```

```
# the correct paragraph was chunk #4,912 and scored 6th
```

06 The whole-book problem: reasoning about what is absent

5:43

RAG retrieves what exists. Some questions are about what is missing, and no snippet contains a gap.

KEY POINTS

- RAG matches queries to passages, and a gap is not a passage.
- Questions about omissions require seeing both documents in full.
- Isolated snippets cannot support a global comparison.
- The same limit applies to counting, exhaustiveness and cross-document consistency.
- If no single passage could contain the answer, top-k retrieval structurally cannot find it.

Why the retrieval succeeds and the answer still fails

Retrieval did its job here — the chunks are genuinely relevant. That is what makes this failure different from silent failure and impossible to fix by improving the embeddings.

```
question: "which security requirements were omitted from the final release?"
```

```
retrieved:
```

```
[1] requirements.md "3.4 All API endpoints shall enforce rate limiting..."
[2] requirements.md "3.7 Session tokens shall expire after 30 minutes..."
[3] release-notes.md "Added rate limiting to public endpoints."
[4] release-notes.md "Improved session handling performance."
```

```
# every chunk is relevant. the answer – that 3.7 was never implemented –
# is in the *absence* of a matching release note, which no chunk contains.
```

07 The case for RAG: you pay to re-read, every single time

7:53

Long context creates a compute inefficiency that scales with traffic. RAG pays the processing cost once, at indexing time.

KEY POINTS

- A 500-page manual is roughly 250,000 tokens, re-processed on every long-context query.
- RAG pays the processing cost once at index time, then a few thousand tokens per query.
- Long-context cost scales as corpus size × query volume; RAG as corpus size + small × volume.
- Prompt caching offsets this only for stable, byte-identical prefixes that stay warm.
- Frequently changing content invalidates the cache and restores the full cost.

Where the two curves cross

Rates are placeholders — substitute your provider's current numbers, since they change. What does not change is the shape: long context is dominated by a term that multiplies corpus size by traffic, and RAG is not.

```
CORPUS_TOKENS = 250_000
RETRIEVED      = 3_000
IN_RATE        = 3.00 / 1_000_000    # per input token
EMBED_RATE     = 0.02 / 1_000_000

def long_context(queries, cache_hit_rate=0.0):
    effective = CORPUS_TOKENS * (1 - cache_hit_rate * 0.9) # cached ≈ 10% cost
    return queries * effective * IN_RATE

def rag(queries):
    index_once = CORPUS_TOKENS * EMBED_RATE
    return index_once + queries * RETRIEVED * IN_RATE

for q in (100, 1_000, 10_000, 100_000):
    print(f"{q:>7,}  long={long_context(q):>9.2f}  "
          f"cached={long_context(q, 0.9):>8.2f}  rag={rag(q):>8.2f}")
```

08 The case for RAG: attention dilutes as context grows

8:36

There is an intuitive assumption that data in the window will be used. Research suggests otherwise.

KEY POINTS

- Attention dilutes as context length grows; presence in the window does not guarantee use.
- Facts buried mid-document are the hardest to surface, and failures can look like hallucination.
- Retrieving the top chunks removes the haystack and hands the model the needles.
- More context can lower accuracy — irrelevant tokens compete with relevant ones.
- Measure position and length effects on your own data; behaviour varies by model and version.

A needle-in-a-haystack test for your own corpus

Insert a fact the model cannot know, at a controlled depth, then ask for it. Sweep length and depth to map where your accuracy falls off — that boundary is what tells you how much context you can actually rely on.

```
NEEDLE = "The internal codename for the Q3 migration is BLUE HERON."
QUESTION = "What is the internal codename for the Q3 migration?"

def trial(filler_tokens, depth_fraction):
    filler = make_filler(filler_tokens)
    cut = int(len(filler) * depth_fraction)
    haystack = filler[:cut] + NEEDLE + filler[cut:]

    out = model.generate(f"{haystack}\n\n{QUESTION}")
    return "BLUE HERON" in out.upper()

for length in (10_000, 50_000, 200_000, 500_000):
    row = [trial(length, d) for d in (0.1, 0.3, 0.5, 0.7, 0.9)]
    print(f"{length:>7,}  " + " ".join("ok " if r else "MISS" for r in row))
```

09 The case for RAG: the infinite dataset

9:19

A million-token window sounds enormous until you compare it to an enterprise data lake measured in terabytes.

KEY POINTS

- Enterprise data lakes are measured in terabytes or petabytes; windows are measured in millions of tokens.
- The gap is structural and is not closing.
- Answering over everything requires a filtering layer that reduces the corpus to what fits.
- The deciding property is whether the corpus is bounded, not how large it is right now.
- A bounded corpus that fits today may not be bounded — check growth, not just current size.

Bounded or unbounded is the real question

Ask what the corpus looks like in two years rather than what it looks like now. Anything that grows with usage — tickets, logs, messages, commits — is unbounded and needs retrieval regardless of today's measurement.

bounded, fits, stays fitting

unbounded, will never fit

one legal contract

every contract the firm has

one book

the company wiki

one service's source

the monorepo

one incident's logs

all logs, all time

a product spec + release notes

the ticket system

10 Choosing

10:05

Bounded data plus global reasoning favours long context. Unbounded enterprise knowledge still requires a vector database.

KEY POINTS

- Bounded corpus plus global reasoning → long context.
- Unbounded enterprise knowledge → retrieval, non-negotiably.
- The common middle ground is retrieve-broadly-then-reason, tuned by k.
- Three measurements decide it: recall on a gold set, needle accuracy by length, cost at real traffic.
- Both defaults have been wrong recently; measure rather than following the trend.

The decision, on one screen

Work down the column that matches your corpus. Most of the disagreement about RAG comes from people arguing from different rows without saying which.

	long context	RAG
corpus	bounded	unbounded
question type	global / comparative	passage-level
cost driver	tokens x every query	one-off indexing
characteristic failure	attention dilution	silent retrieval miss
infrastructure	none	embed + store + sync
freshness	re-read the file	re-index the document
answers "what is missing?"	yes	no
scales to a data lake	no	yes

Glossary

Context injection

Getting the right external data into a model's prompt at query time, so it can answer about things outside its training data.

Context window

The maximum number of tokens a model can process in one call, covering the prompt and the generated output together.

Chunking

Splitting documents into smaller passages before embedding. Strategies include fixed size, sliding window and recursive splitting on structure.

Embedding model

A model that converts text into a vector whose position encodes meaning, so similar passages sit near each other.

Vector database

Storage built for fast nearest-neighbour search over embeddings, plus the indexing and sync machinery around it.

Semantic search

Ranking documents by vector similarity to the query rather than by shared keywords, so meaning matches even when wording differs.

Silent failure

When retrieval misses a document that contained the answer. No error is raised — the model simply answers from the wrong material, confidently.

Reranker

A second-stage model that re-scores retrieved candidates more accurately than vector similarity, usually over a shortlist.

Long context

Placing whole documents directly in the prompt and letting the attention mechanism locate what matters, with no retrieval step.

Prompt caching

Reusing a provider's processed representation of an identical prompt prefix across requests, cutting the cost of the repeated portion.

Needle in a haystack

A test that plants a known fact at a controlled depth in a long context and checks whether the model can retrieve it.

Attention dilution

The degradation in a model's ability to focus on a specific passage as the surrounding context grows.

recall@k

The fraction of questions for which the correct source passage appears in the top k retrieved results. The ceiling on a RAG system's accuracy.

Token

The unit a model actually processes — a word fragment. Roughly three-quarters of a word for English prose, and much denser for code or JSON.

Check yourself

Answer first, then open each one.

Why is a retrieval miss described as a silent failure, and what does that imply about monitoring?

Because nothing errors: retrieval returns plausible chunks, the model answers fluently from them, and the response is a 200 with no warning. Standard monitoring cannot see it, since there is no failure signal to alert on. The only way to detect it is a gold set of questions with known source passages and a recall@k measurement run deliberately.

Retrieval returns four chunks that are all genuinely relevant, yet the system still cannot answer 'which requirements were dropped from the release?'. Why is better embedding not the fix?

The answer is an absence, and an absence is not a passage — there is no text anywhere in the corpus that says a requirement was omitted, so no embedding can point at it. The question requires holding both documents in full and comparing them. This is a structural limit of passage retrieval, not a quality problem.

A team enables prompt caching for a long-context application and sees no cost reduction. What is the most likely cause?

Something variable sits at the front of the prompt — a timestamp, a session ID, the question itself — so the cacheable prefix differs on every request and the cache misses every time. Caching keys on an exact prefix, so all stable bulk content must come first and everything variable must come last.

Why can adding more relevant-looking context to a prompt make answers worse rather than better?

Attention dilutes as context grows: irrelevant material competes with relevant material for the model's focus, and a fact buried mid-document can be missed or blended with surrounding text. This is why retrieving the top few chunks often beats supplying everything, even when everything fits and you are willing to pay for it.

Your corpus is 400,000 tokens today and fits comfortably in the window. Why might retrieval still be the right architecture?

Because the deciding property is whether the corpus is bounded, not its size right now. If it grows with usage — tickets, logs, commits, messages — today's fit is temporary, and rebuilding the data path later is far more expensive than building retrieval now. Ask what it looks like in two years.

Why does the hybrid pattern — retrieve fifty chunks, then reason over all of them — capture most of both arguments?

A high k makes silent failure unlikely, because the correct passage almost certainly appears somewhere in a wide net. Bounding the result to a moderate context keeps attention sharp and cost predictable, while letting the model do global reasoning over what survived. Retrieval acts as a coarse filter rather than a precise selector, which is the job it is actually good at.

Where to go next

- Build a gold set of 30 questions with known source passages for your own corpus and plot $\text{recall}@k$ for k in 1, 3, 5, 10, 25, 50 — the plateau is the ceiling on any RAG system you build on it.
- Run the needle-in-a-haystack script against the model you actually use, sweeping length and depth, and find where accuracy falls off for your content type rather than for generic prose.
- Compute the crossover point where long-context cost exceeds RAG cost at your real query volume, then recompute it with a realistic prompt-cache hit rate.
- Take a question your RAG system answers badly and re-ask it with the full documents in context; if it becomes correct, you have found a whole-book problem rather than a retrieval-quality problem.
- Implement the retrieve-60-then-reason hybrid and sweep k against both recall and answer accuracy — the two curves usually peak at different values, and the gap is informative.
- Instrument your RAG pipeline to log the retrieved chunk IDs alongside every answer, so that when a wrong answer is reported you can tell immediately whether retrieval or generation was at fault.

Lesson generated from a YouTube transcript with YouTube Lesson Builder.

Explanations are AI-generated. Check anything you intend to rely on.